

Page Replacements Algorithms

CODE:

```
#include <iostream>
#include <vector>
#include <queue>
#include <unordered_map>
using namespace std;

// Function to print frames
void printFrames(vector<int> &frames) {
    for (int f : frames) {
        if (f == -1) cout << "- ";
        else cout << f << " ";
    }
}

// ===== FCFS =====
int FCFS(vector<int> pages, int capacity) {
    vector<int> frames(capacity, -1);
    queue<int> q;
    int faults = 0;

    cout << "\n--- FCFS ---\n";

    for (int page : pages) {
        bool found = false;

        for (int f : frames) {
            if (f == page) {
                found = true;
                break;
            }
        }

        if (!found) {
```

```

        faults++;
        if (q.size() < capacity) {
            q.push(page);
            frames[q.size() - 1] = page;
        } else {
            int old = q.front();
            q.pop();

            for (int i = 0; i < capacity; i++) {
                if (frames[i] == old) {
                    frames[i] = page;
                    break;
                }
            }
            q.push(page);
        }

        printFrames(frames);
        cout << " *\n";
    } else {
        printFrames(frames);
        cout << "\n";
    }
}

cout << "Total Page Faults (FCFS): " << faults << endl;
return faults;
}

// ===== LRU =====
int LRU(vector<int> pages, int capacity) {
    vector<int> frames;
    unordered_map<int, int> recent;
    int faults = 0;

    cout << "\n--- LRU ---\n";

    for (int i = 0; i < pages.size(); i++) {
        int page = pages[i];
        bool found = false;

```

```
for (int f : frames) {
    if (f == page) {
        found = true;
        break;
    }
}

if (!found) {
    faults++;

    if (frames.size() < capacity) {
        frames.push_back(page);
    } else {
        int lru_page = frames[0];
        int min_index = recent[lru_page];

        for (int f : frames) {
            if (recent[f] < min_index) {
                min_index = recent[f];
                lru_page = f;
            }
        }

        for (int i = 0; i < frames.size(); i++) {
            if (frames[i] == lru_page) {
                frames[i] = page;
                break;
            }
        }
        printFrames(frames);
        cout << " *\n";
    } else {
        printFrames(frames);
        cout << "\n";
    }

    recent[page] = i;
}
```

```
    cout << "Total Page Faults (LRU): " << faults << endl;
    return faults;
}

// ===== OPTIMAL =====
int Optimal(vector<int> pages, int capacity) {
    vector<int> frames;
    int faults = 0;

    cout << "\n--- Optimal ---\n";

    for (int i = 0; i < pages.size(); i++) {
        int page = pages[i];
        bool found = false;

        for (int f : frames) {
            if (f == page) {
                found = true;
                break;
            }
        }

        if (!found) {
            faults++;

            if (frames.size() < capacity) {
                frames.push_back(page);
            } else {
                int index = -1, farthest = i;

                for (int j = 0; j < frames.size(); j++) {
                    int k;
                    for (k = i + 1; k < pages.size(); k++) {
                        if (frames[j] == pages[k]) {
                            if (k > farthest) {
                                farthest = k;
                                index = j;
                            }
                        }
                    }
                    break;
                }
            }
        }
    }
}
```

Name : Raman Randive
Course : Operating System

PRN: 123B1B240
Assignment : Page Replacement algorithms

```
    }  
    }  
  
    if (k == pages.size()) {  
        index = j;  
        break;  
    }  
}  
  
if (index == -1) index = 0;  
frames[index] = page;  
}  
  
printFrames(frames);  
cout << " *\n";  
} else {  
    printFrames(frames);  
    cout << "\n";  
}  
}  
  
cout << "Total Page Faults (Optimal): " << faults << endl;  
return faults;  
}  
  
// ===== MAIN =====  
int main() {  
    int n, capacity;  
  
    cout << "Enter number of pages: ";  
    cin >> n;  
  
    vector<int> pages(n);  
    cout << "Enter reference string: ";  
    for (int i = 0; i < n; i++) {  
        cin >> pages[i];  
    }  
  
    cout << "Enter number of frames: ";  
    cin >> capacity;
```

Name : Raman Randive
Course : Operating System

PRN: 123B1B240
Assignment : Page Replacement algorithms

```
int f1 = FCFS(pages, capacity);  
int f2 = LRU(pages, capacity);  
int f3 = Optimal(pages, capacity);  
  
cout << "\n--- Comparison ---\n";  
cout << "FCFS Faults   : " << f1 << endl;  
cout << "LRU Faults     : " << f2 << endl;  
cout << "Optimal Faults : " << f3 << endl;  
  
return 0;  
}
```

Name : Raman Randive

PRN: 123B1B240

Course : Operating System

Assignment : Page Replacement algorithms

```
PS C:\Users\raman\Desktop\COLLEGE ASSIGNMENT\sem 6\os\Assignment 4> cd -c:\Users\raman\Desktop\COLLEGE ASSIGNMENT\sem 6\os\Assignment 4\" ; if ($?) { g++ code.cpp -o code } ; if ($?) { .\code }
Enter number of pages: 15
Enter reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 14
Enter number of frames: 4

--- FCFS ---
7 - - - *
7 0 - - *
7 0 1 - *
7 0 1 2 *
7 0 1 2
3 0 1 2 *
3 0 1 2
3 4 1 2 *
3 4 1 2
3 4 1 2
3 4 0 2 *
3 4 0 2
3 4 0 2
3 4 0 1 *
14 4 0 1 *
Total Page Faults (FCFS): 9

--- LRU ---
7 *
7 0 *
7 0 1 *
7 0 1 2 *
7 0 1 2
3 0 1 2 *
3 0 1 2
3 0 4 2 *
3 0 4 2
3 0 4 2
3 0 4 2
3 0 4 2
3 0 4 2
3 0 1 2 *
3 14 1 2 *
Total Page Faults (LRU): 8

--- Optimal ---
7 *
7 0 *
7 0 1 *
7 0 1 2 *
7 0 1 2
3 0 1 2 *
3 0 1 2
3 0 4 2 *
3 0 4 2
3 0 4 2
3 0 4 2
3 0 4 2
3 0 4 2
1 0 4 2 *
14 0 4 2 *
Total Page Faults (Optimal): 8

--- Comparison ---
FCFS Faults : 9
LRU Faults : 8
Optimal Faults : 8
PS C:\Users\raman\Desktop\COLLEGE ASSIGNMENT\sem 6\os\Assignment 4>
```